

소프트웨어공학

객체지향 방법론 1

- 1 객체지향의 배경
- 2 객체지향 기법과 절차지향 기법에 대한 비교
- 3 객체지향 기법의 구성요소

01

객체지향의 배경

- 과학, 군사적 모의실험(simulation)을 위해 컴퓨터를 이용한 가상세계를 구현하려는 노력으로부터 객체지향이론이 시작됨
- 객체지향 기법은 절차지향 기법(구조적 기법)의 문제점으로 인한 소프트웨어 위기의 해결책으로 채택되어 사용되었다.
- 컴퓨팅 환경에 대한 보다 많은 기능, 단순성, 재사용성, 사용 편의성 등에 대한 사용자 요구가 증대되었다.
- 1995년 말 Java탄생. 객체지향언어가 프로그래밍 언어의 주류가 됨.

02

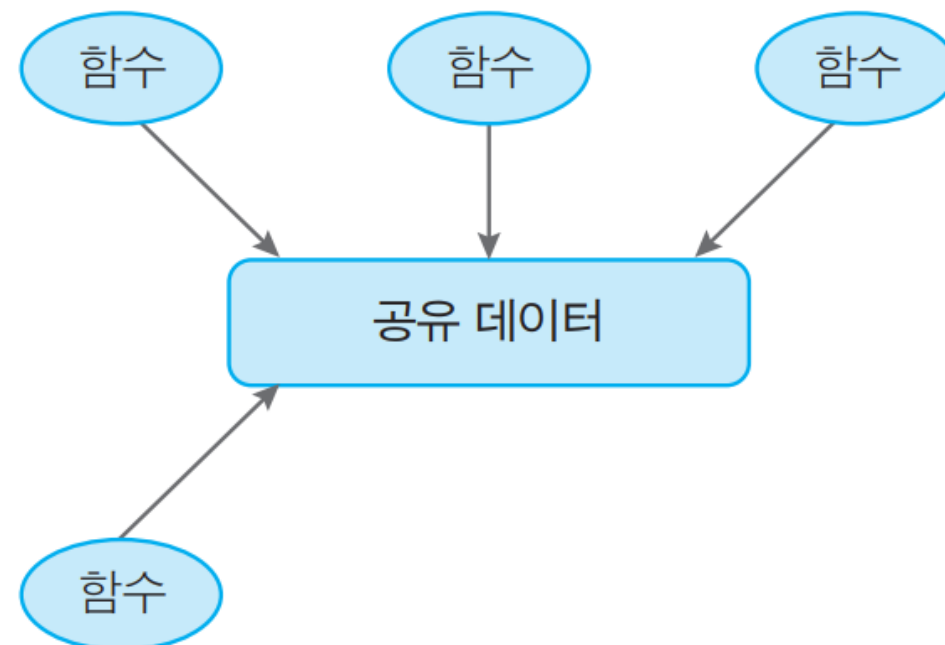
객체지향 기법과 절차지향 기법에 대한 비교

- 객체지향 대 절차지향

절차지향 (object-oriented)

하나의 커다란 작업을 여러 개의 작은 작업으로 분할하고, 분할된 각각의 소작업을 수행하는 함수 (모듈)를 작성하여, 프로그램의 절차에 따라 함수를 호출하는 형태로 진행되는 기법이다. 데이터와 그 데이터를 변화시키는 함수로 구성되어 있다.

절차지향 : 프로그램 = 데이터 + 함수



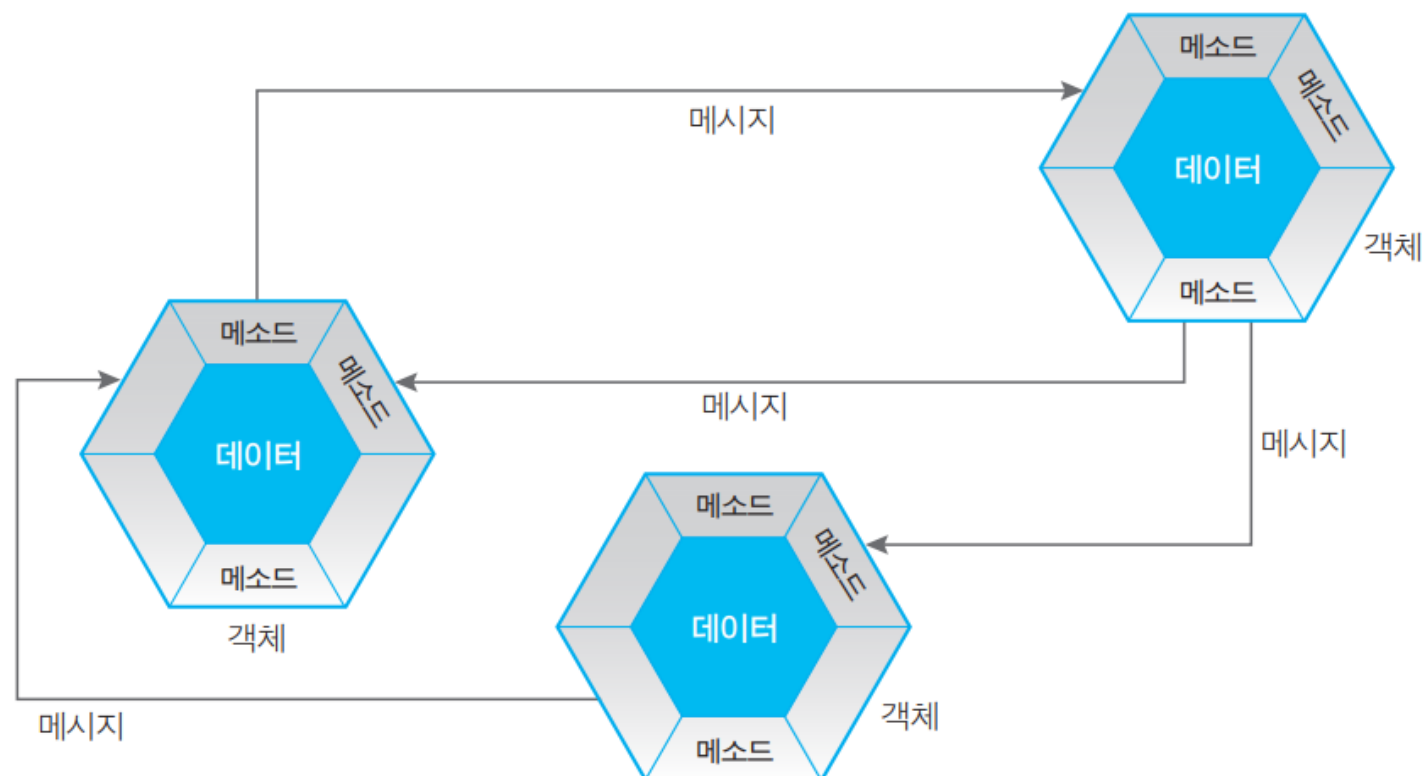
객체지향 기법과 절차지향 기법에 대한 비교

- 객체지향 대 절차지향

객체지향 (procedural- Oriented)

현실 세계의 개체(entity)를 기계의 부품처럼 하나의 객체로 만들어, 기계적인 부품들을 조립하여 제품을 만들 듯이 소프트웨어를 개발할 때에도 객체들을 조립해서 작성할 수 있도록 하는 기법이다.
실세계의 개체를 속성(attribute)과 메소드(method)가 결합된 형태의 객체로 표현하는 개념이다.

[객체 = 데이터 + 메소드(함수)] [프로그램 = 객체 + 객체]



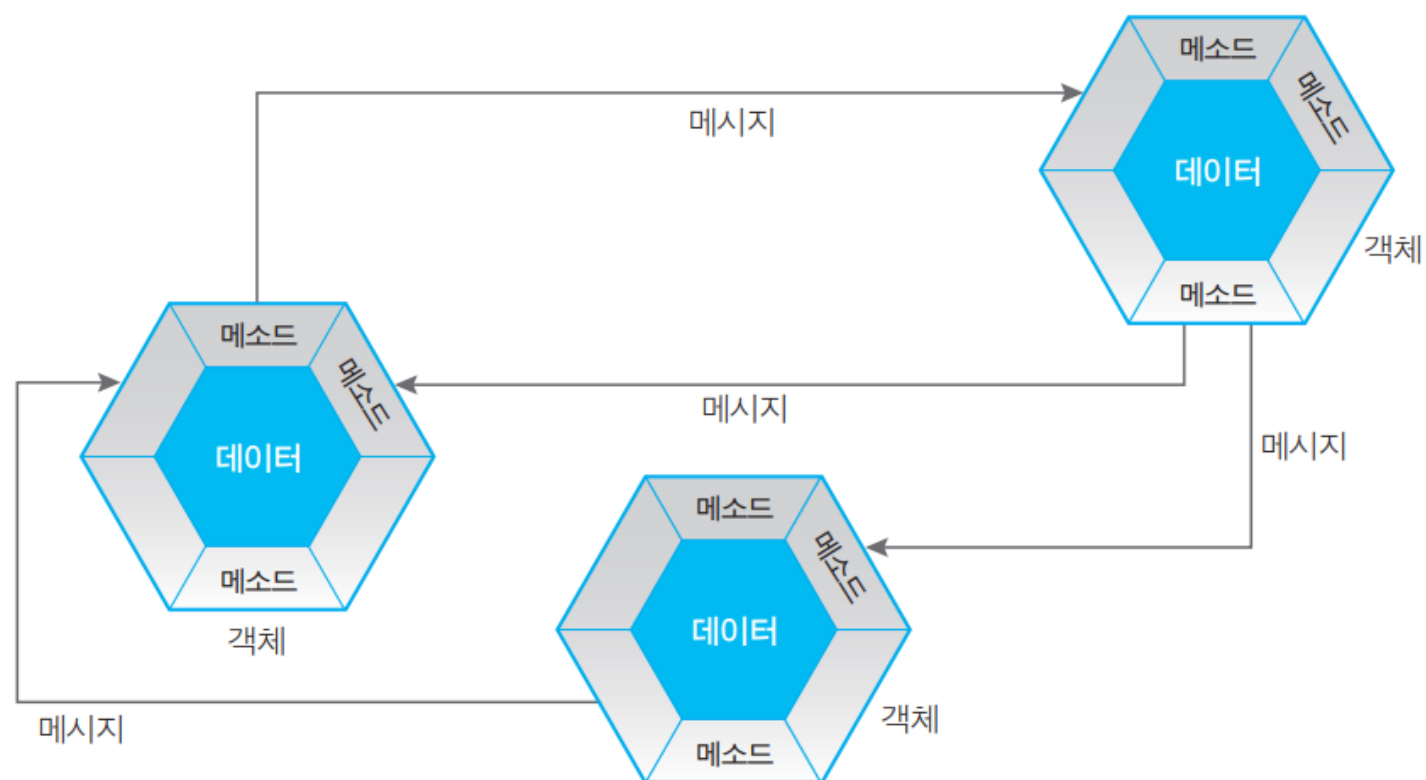
객체지향 기법과 절차지향 기법에 대한 비교

- 객체지향 대 절차지향

객체지향의 장점

1. 현실 세계를 모형화하여 사용자와 개발자가 쉽게 이해 할 수 있다.
2. 자연적인 모델링이 가능하다.
3. 소프트웨어 재사용률이 높아진다.
4. 소프트웨어 유지보수성이 향상된다.

[객체 = 데이터 + 메소드(함수)] [프로그램 = 객체 + 객체]



객체지향 기법과 절차지향 기법에 대한 비교

- 객체지향 언어의 특징
 - 기존의 프로그래밍언어와 크게 다르지 않다.
 - 기존의 프로그래밍 언어에 몇가지 규칙을 추가한 것일 뿐이다.
 - 코드의 재사용성이 높다.
 - 새로운 코드를 작성할 때 기존의 코드를 이용해서 쉽게 작성할 수 있다.
 - 코드의 관리가 쉬워졌다.
 - 코드간의 관계를 맷어줌으로써 보다 적은 노력으로 코드변경이 가능하다.
 - 신뢰성이 높은 프로그램의 개발을 가능하게 한다.
 - 제어자와 메서드를 이용해서 데이터를 보호하고, 코드의 중복을 제거하여 코드의 불일치로 인한 오류를 방지할 수 있다.

객체지향 기법과 절차지향 기법에 대한 비교

- 객체지향 언어의 특징
 - 객체지향언어(장점)
 - 재사용성: 상속을 통해 프로그래밍시 코드의 재사용향상
 - 생산성 향상: 잘 설계된 클래스를 만들어서 독립적인 객체를 사용함으로써 개발의 생산성 향상
 - 자연적인 모델링: 우리 일상생활의 모습의 구조가 그대로 사용하기 때문에 자연스럽게 구현
 - 유지보수의 우수성: 프로그램 수정시 추가, 수정을 하더라도 캡슐화를 통해 주변 영향이 적기때문에 유지보수가 쉬워서 매우 경제적

객체지향 기법과 절차지향 기법에 대한 비교

- 객체지향언어의 특징
 - 객체지향언어(단점)
 - 개발속도가 느린점: 객체의 정확한 이해가 필요하기에 설계단계에 많은 시간 소모
 - 실행속도가 느린점: 객체지향언어는 대체적으로 실행속도가 느림
 - 코딩난이도 상승: 다중 상속이 지원등과 같이 복잡한 코딩으로 난이도 상승

03

객체지향 기법의 구성요소

- 클래스, 객체, 메서드



이미지 확인 필요합니다

03

객체지향 기법의 구성요소

- 객체

객체(Object)	- 실세계(real world)에서 어떤 구체적 의미를 구성하는 하나의 실체 단위로 특정 사물 및 개념이다. - 필요로 하는 데이터와 그 위에 수행되는 메소드를 묶어 놓은(캡슐화한) 소프트웨어 모듈이다.
데이터(Attribute)	- 객체가 가지고 있는 정보로 속성이나 상태, 분류 등을 나타낸다. - 속성(attribute), 상태, 변수, 상수, 자료 구조라고도 한다.
메소드 (함수)	- 객체가 수행하는 기능으로 객체가 갖는 데이터 (속성, 상태)를 처리하는 알고리즘 - 객체의 상태를 참조하거나 변경하는 수단이 되는 것으로 메소드(method), 서비스(service), 동작(operation), 연산이라고도 한다.

03

객체지향 기법의 구성요소

- 객체

객체의 특성

- 객체는 속성과 행동을 가지고 있으며 객체의 행동은 자신이 수행하는 오퍼레이션(operation)으로 구성된다.
- 객체는 상태를 유지할 수 있어야 하며 기능을 가져야 한다.
- 객체는 메소드는 다른 객체로부터 메시지를 받았을 때 수행된다.

03

객체지향 기법의 구성요소

- 클래스

클래스

- 객체는 항상 클래스로부터 생성된다. 즉 클래스는 객체를 생성하는 형판이다.
- 클래스는 2개의 구성요소인 자료구조(필드)와 연산(메소드)을 가진다.
- 클래스로부터 생성된 객체를 인스턴스라 하며, 클래스로부터 새로운 객체를 생성하는 것을 인스턴스화라고 한다.
- 정보처리의 주체는 클래스가 아니라 객체이다.
- 객체지향 프로그래밍의 시작은 클래스의 생성이다.
- 최상위 클래스는 상위 클래스를 갖지 않는 유일한 클래스를 의미한다.

- 클래스로부터 객체의 생성 예(Java 언어)

학생 1 = New 학생 (이기쁨, 남자, 컴퓨터 공학과, 2학년)

↓ ↓ ↓ ↓

객체 객체생성 클래스 매개변수
이름 명령어 이름 데이터

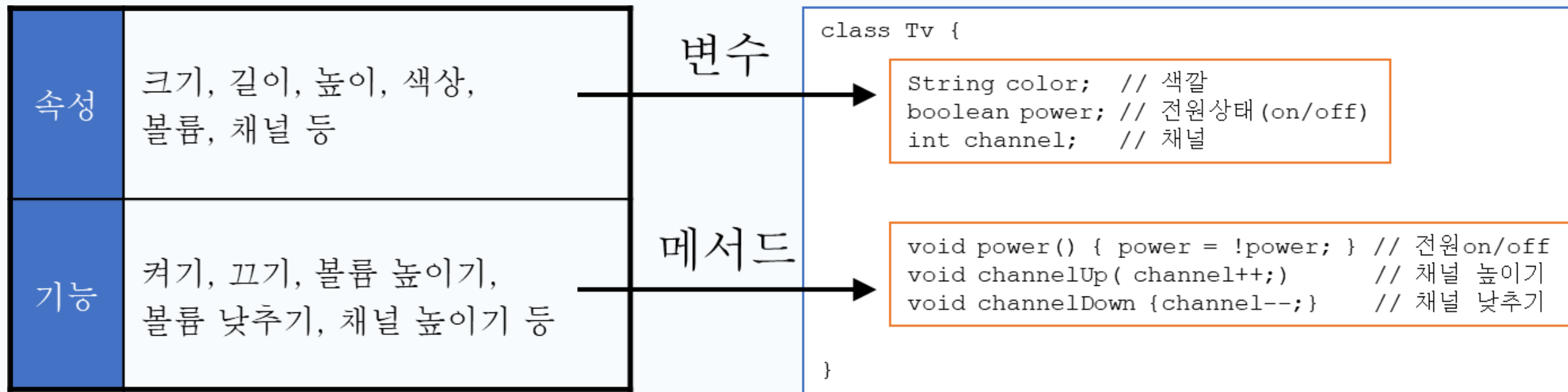
03

객체지향 기법의 구성요소

- 객체

객체

- 객체의 구성요소 - 속성과 기능
- 객체는 속성과 기능으로 이루어져 있다.
→ 객체는 속성과 기능의 집합이며, 속성과 기능을 객체의 멤버(member, 구성요소)라고 한다.
- 속성은 변수로, 기능은 메서드로 정의한다.
→ 클래스를 정의할 때 객체의 속성은 변수로, 기능은 메서드로 정의한다.



03

객체지향 기법의 구성요소

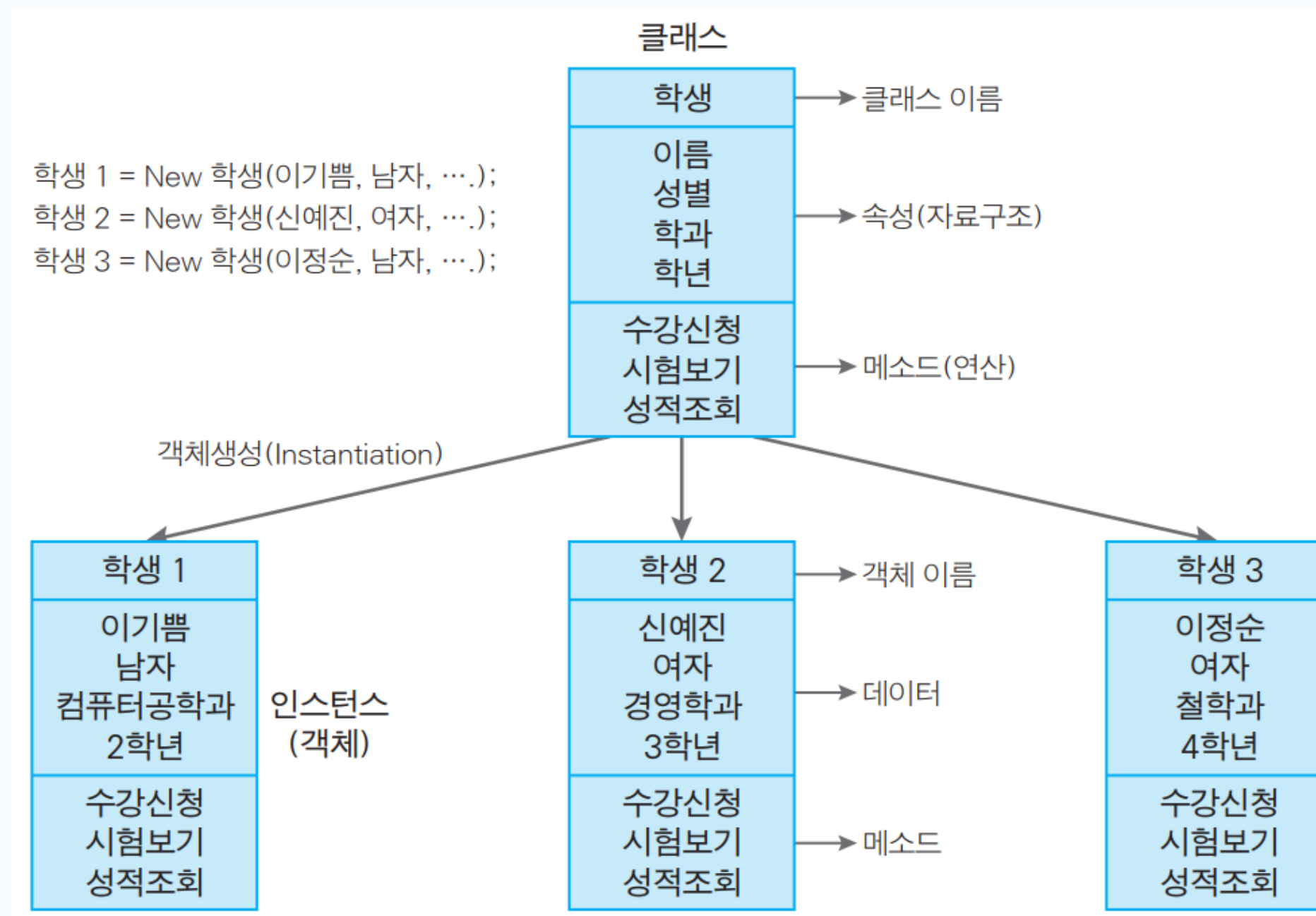
- 메시지

메시지	• 메시지는 객체들 간에 상호작용을 하는데 사용되는 수단으로, 객체의 메소드를 호출하는 외부의 요구사항이다.
메시지 구성요소	• 메시지를 받을 객체의 이름(주소) • 송신객체가 실행을 원하는 수신객체의 메소드 이름 • 실행을 원하는 메소드에 전달할 매개변수 (인자, 필요할 때만 사용)

03

객체지향 기법의 구성요소

- 클래스 다이어그램

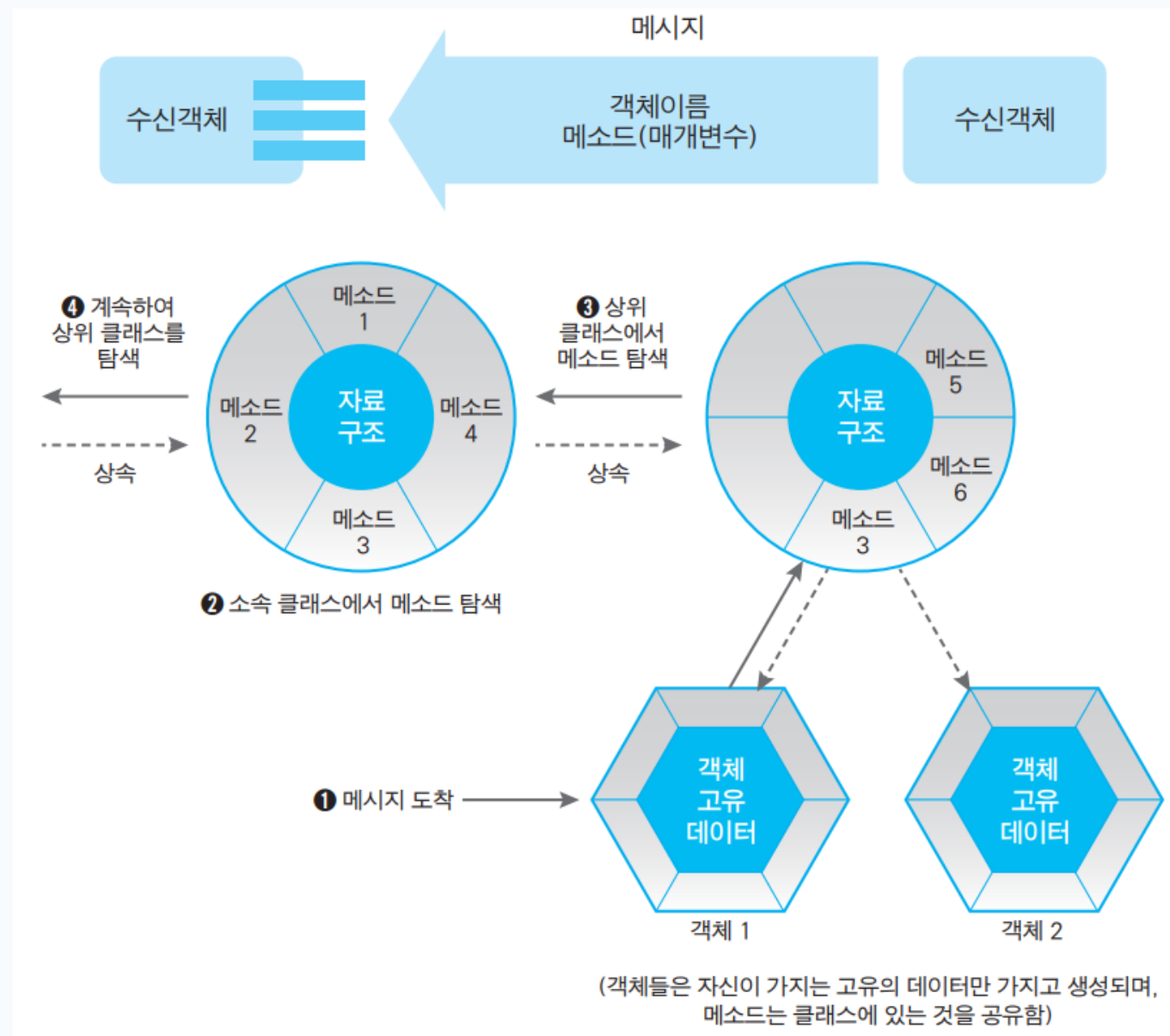


03

객체지향 기법의 구성요소

- 메시지

- 메시지를 받은 수신객체는 요구된 메소드를 수행하여 결과를 반환한다.



- 클래스 계층 구조에서의 메시지 처리